Vienna Traffic Router

Complete Study Notes: Algorithms, Heuristics, Cost Functions, Data Flow

AI Search Algorithms with Real-World Heuristics on OpenStreetMap

Vienna Traffic Router — AI Search Algorithms Project

April 2026

Contents

1 Project Overview **2**
- 1.1 Technology Stack . . . 2

2 System Architecture — Data Flow **2**
- 2.1 Key Source Files . . . 2

3 Graph Structure **2**
- 3.1 Nodes and Edges . . . 2
- 3.2 District Lookup . . . 3
- 3.3 Reverse Edges . . . 3

4 Distance Measurement: Haversine Formula **3**

5 Edge Cost — $g(n)$ **3**
- 5.1 Effective Cost Formula . . . 3
- 5.2 Vehicle Passability . . . 4

6 Heuristic Function — $h(n)$ **4**
- 6.1 Core Formula . . . 4
- 6.2 Three-Tier Architecture . . . 5
  - 6.2.1 Tier 1 — Static (Computed Once at Closure Build) . . . 5
  - 6.2.2 Tier 2 — Goal-Dependent (Constant per Route) . . . 5
  - 6.2.3 Tier 3a — Node-Dependent (Per Frontier Node) . . . 5
  - 6.2.4 Tier 3b — Edge-Dependent (Cheapest Outgoing Edge Proxy) . . . 6

7 All 30 Heuristic Functions — Reference **6**
- 7.1 Environmental (5) . . . 6
- 7.2 Temporal (5) . . . 6
- 7.3 Vehicle-Type (3) . . . 7
- 7.4 Road Geometry & Condition (6) . . . 7
- 7.5 Safety & Regulation (6) . . . 7
- 7.6 Urban Features & Attractions (5) . . . 7
- 7.7 Destination-Specific (2) . . . 7

8 Admissibility, Capping, and ε-Admissibility **7**
- 8.1 What is Admissibility? . . . 7
- 8.2 The MAX_HEURISTIC_CAP . . . 8
- 8.3 ε-Admissibility (Weighted A*) . . . 8

9 The 8 Search Algorithms **8**
9.1 How All Algorithms Share the Same Interface . . . 8
9.2 Algorithm 1: BFS (Breadth-First Search) . . . 8
9.3 Algorithm 2: DFS (Depth-First Search) . . . 8
9.4 Algorithm 3: UCS (Uniform-Cost Search) . . . 8
9.5 Algorithm 4: Dijkstra . . . 9
9.6 Algorithm 5: Greedy Best-First Search . . . 9
9.7 Algorithm 6: A* . . . 9
9.8 Algorithm 7: Weighted A* . . . 9
9.9 Algorithm 8: Bidirectional A* . . . 9

10 How Heuristics Affect Each Algorithm **10**

11 Parallel Runner and PathResult **10**
11.1 run_all() . . . 10
11.2 PathResult Fields . . . 11

12 Closure Factory Pattern — Performance **11**
12.1 Why a Closure? . . . 11
12.2 enable/disable via UI . . . 11

13 Fully Worked Heuristic Calculation Example **12**

14 Algorithm Performance Summary **13**

15 Quick Q&A: Likely Professor Questions **13**

16 Summary of All Heuristic Functions (Tier Classification) **14**

1 Project Overview

Vienna Traffic Router runs **8 classic AI search algorithms** in parallel on a curated subgraph of Vienna's OpenStreetMap road network (≈1 000 nodes, ≈10 000 edges), shows all 8 resulting routes on a Leaflet map, and lets the user tune 30+ heuristic parameters (weather, time of day, vehicle type, and Vienna-specific local effects) to watch the routes change in real time.

> **Core Idea.** Same graph, same start/goal, 8 different search strategies — observe how heuristics affect which roads are chosen, how many nodes are explored, and how far each route is from optimal.

1.1 Technology Stack

- **Backend:** FastAPI (Python), Pydantic, ThreadPoolExecutor
- **Graph data:** OpenStreetMap via Overpass API → JSON adjacency list
- **Algorithms:** BFS, DFS, UCS, Dijkstra, Greedy, A*, Weighted A*, Bidirectional A*
- **Frontend:** Vanilla JS, Leaflet.js, dark "Imperial Vienna" theme
- **Heuristics:** 30 pluggable multiplier modules, closure-factory pattern

2 System Architecture — Data Flow

Understanding the data flow answers: *"Where does a click on the map become a route on screen?"*

`Browser click` $\xrightarrow{\text{lat/lon + 30 params}}$ **POST /api/find-path**
↓ Pydantic `FindPathRequest` validates payload
make_heuristic(params) builds closure (static tier computed once)
↓ heuristic_fn passed to runner
run_all() → ThreadPoolExecutor(8 threads) → 8 algorithms in parallel
↓ each thread calls `find_path(graph, start, goal, h_fn, params)`
Dijkstra result used as **ground truth** → compute `optimality_gap_pct`
↓ 8 PathResult dicts returned
JSON response → browser draws 8 routes, comparison table

Figure 1: End-to-end request data flow

2.1 Key Source Files

File	Responsibility
`src/main.py`	FastAPI app entry point, static file serving
`src/routes/api.py`	`FindPathRequest` Pydantic model, `_overrides` dict, POST handler
`src/heuristics/combined.py`	Closure factory `make_heuristic()`, all 30 heuristic imports
`src/algorithms/runner.py`	Parallel execution, nearest-node snapping, Dijkstra ground truth
`src/algorithms/base.py`	`PathResult`, `sum_path_distance()`, `VEHICLE_SPEEDS`
`src/graph/builder.py`	OSM parsing, node curation, edge enrichment, `haversine()`
`src/graph/loader.py`	Graph singleton, `get_effective_edge_cost()`

3 Graph Structure

3.1 Nodes and Edges

- **Nodes**: ≈1 000 curated Vienna OSM nodes (intersections, waypoints). Selected via a 7-tier priority curation to keep the graph small but representative.

- **Edges**: ≈10 000 bidirectional edges. Each edge is a dict with rich metadata.
- **Adjacency list**: `adjacency[node_id] = [{"node": nb_id, "edge_idx": int}, ...]`
- **Edge fields**: `from`, `to`, `distance_m`, `surface`, `road_type`, `elevation_from`, `elevation_to`, `lit`, `tram_track`, `bridge`, `tunnel`, `near_school`, `near_green_space`, `snow_priority`, `motor_vehicle`, `lanes`, ...

Why edge_idx?. The flat `edges` list is shared across all algorithms. An `edge_idx` pointer into this list is cheaper to store in the adjacency entry than a full copy of the edge dict. Never flatten this structure.

3.2 District Lookup

`get_district(lat, lon)` in `builder.py` uses bounding-box lookup for Vienna's 23 districts. Several heuristics key off district strings (e.g. parking, events, pedestrian density). "unknown" means "don't apply the district penalty".

3.3 Reverse Edges

`build_graph` emits a **reverse edge for every non-strictly-oneway way**. This is load-bearing for Bidirectional A*: the backward search's predecessors are found by scanning forward-adjacency of the goal side. One-way streets correctly have no reverse edge.

4 Distance Measurement: Haversine Formula

All distances use the **Haversine formula** — great-circle distance over Earth's surface:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{1}$$

$$d = 2R \cdot \operatorname{atan2}\left(\sqrt{a},\, \sqrt{1-a}\right), \quad R = 6\,371\,000 \text{ m} \tag{2}$$

Unit: all distances are in **metres**. $g(n)$ and $h(n)$ share the metre unit — never mix with seconds.

```python
def haversine(lat1, lon1, lat2, lon2) -> float:
    R = 6_371_000
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = math.sin(dphi/2)**2 + math.cos(phi1)*math.cos(phi2)*math.sin(dlam/2)**2
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))
```

Listing 1: src/graph/builder.py — haversine()

5 Edge Cost — $g(n)$

$g(n)$ = cumulative sum of **effective edge costs** from start to node n.

5.1 Effective Cost Formula

$$c_{\text{eff}}(e) = \underbrace{d_e}_{\text{distance_m}} \times \underbrace{m_{\text{override}}(e)}_{\text{manual block}} \times \underbrace{m_{\text{road_quality}}(e, v)}_{\text{road type/surface}}$$

Manual Override Multiplier

$$m_{\text{override}}(I) = \begin{cases} \infty & I \geq 95 \quad \text{(edge impassable)} \\ 0.5 + \dfrac{I}{100} \cdot 2.5 & I < 95 \end{cases}$$

Range: [0.5, 2.85]. Override intensity I is a UI slider set by the user on any edge.

Road Quality Multiplier (g-cost) Applied to $g(n)$ only (not just heuristic), making primary roads faster and footways slower:

$$T(\text{primary}) = 0.55 \quad T(\text{secondary}) = 0.65 \quad T(\text{tertiary}) = 0.80$$
$$T(\text{residential}) = 1.00 \quad T(\text{footway}) = 1.50 \quad T(\text{steps}) = 2.50$$

```python
def get_effective_edge_cost(edge, overrides_map=None, params=None):
    cost = edge["distance_m"]
    if overrides_map:
        key = f'{edge["from"]}_{edge["to"]}'
        if key in overrides_map:
            intensity = overrides_map[key]
            if intensity >= 95:
                return float("inf")
            cost *= 0.5 + (intensity / 100.0) * 2.5
    if params is not None and params.get("use_road_quality", True):
        from src.heuristics.road_quality import get_road_quality_multiplier
        cost *= get_road_quality_multiplier(edge, params.get("vehicle_type"))
    return cost
```

Listing 2: src/graph/loader.py — get_effective_edge_cost()

5.2 Vehicle Passability

`is_edge_passable(edge, vehicle_type)` completely **removes** edges for incompatible vehicle types:

- Car/taxi/truck: primary, secondary, residential roads
- Bicycle/escooter: cycleway + roads
- Walking: footway + residential + parks
- Metro/train: **0 passable edges** (impassable everywhere; comparison only)

6 Heuristic Function — $h(n)$

$h(n)$ = **estimate** of remaining cost from n to goal. Used by Greedy, A*, Weighted A*, Bidirectional A*.

6.1 Core Formula

$$\boxed{h(n, g;\, \theta) = \min\big(\, d_0(n, g) \cdot M(n, g; \theta),\ C \cdot d_0(n, g)\,\big)}$$

where $d_0 = \text{haversine}(n, g)$, $C = \texttt{MAX_HEURISTIC_CAP} = 50$, and the composite multiplier is:

$$M = \underbrace{\prod_{k \in \mathcal{S}} m_k(\theta)}_{\text{static (once)}} \cdot \underbrace{\prod_{k \in \mathcal{G}} m_k(g, \theta)}_{\text{goal-dependent}} \cdot \underbrace{\prod_{k \in \mathcal{N}} m_k(n, G, \theta)}_{\text{node-dependent}} \cdot \underbrace{\min_{e \in \text{adj}(n)} \prod_{k \in \mathcal{D}} m_k(e, \theta)}_{\text{edge proxy (cheapest)}}$$

Why multiplicative, not additive?. Multiplicative composition means a road penalized by weather ($\times 1.5$), surface ($\times 1.2$), and elevation ($\times 1.1$) incurs combined penalty $1.5 \times 1.2 \times 1.1 = 1.98$. Additive would give $0.5 + 0.2 + 0.1 = 0.8$ — no magnitude scaling. Multiplicative correctly scales the Haversine base distance.

6.2 Three-Tier Architecture

6.2.1 Tier 1 — Static (Computed Once at Closure Build)

These depend only on request params, not on which node we're evaluating. Baked into a single `static_mult` float *before* pathfinding begins.

$\mathcal{S} = \{$`weather`, `time_of_day`, `vehicle_type`, `visibility`, `humidity`, `nightnetwork`, `wind`, `headway`$\}$

Heuristic	Input	Range
`weather`	condition string → lookup table	1.0 (clear) . . . 2.0 (black ice)
`time_of_day`	hour + day-of-week	0.65 (weekend night) . . . 1.70 (PM rush)
`vehicle_type`	vehicle string	0.40 (metro) . . . 1.80 (walking)
`visibility`	metres of visibility	1.00 (≥3000 m) . . . 1.60 (<300 m)
`humidity`	relative %	1.0 . . . 1.10 (high humidity + exposed vehicle)
`nightnetwork`	vehicle + hour	1.00 or 1.30 (bus/metro 0–5 AM)
`wind`	speed m/s + direction	1.0 . . . ∼1.15 (headwind, bicycle)
`headway`	vehicle + peak flag	1.0 . . . 1.60 (train off-peak)

```python
static_mult = 1.0
if _enabled(enabled, "weather"):
    static_mult *= get_weather_multiplier(weather, vehicle)
if _enabled(enabled, "time_of_day"):
    static_mult *= get_time_multiplier(hour, day_of_week)
if _enabled(enabled, "vehicle_type"):
    static_mult *= get_vehicle_multiplier(vehicle)
# ... visibility, humidity, nightnetwork, wind, headway
```

Listing 3: combined.py — static tier (computed once)

6.2.2 Tier 2 — Goal-Dependent (Constant per Route)

Depend on the *destination district* — the same for all nodes during one route search.

$\mathcal{G} = \{$`parking`, `holiday_historical`, `events`, `fiaker`, `season`, `pedestrian_density`, `delivery`$\}$

Heuristic	Example value
`parking`	District 1 (Innerstadt) car: 1.60 (very hard to park)
`holiday_historical`	Austrian public holiday, District 1: 1.20; outer: 0.80
`events`	Vienna Marathon (zones 1–4,9): 2.50
`fiaker`	District 1, car, 10–18h: 1.25 (horse carriages)
`season`	November–December, District 1 (Advent): 1.25
`pedestrian_density`	District 1, bicycle, 11–20h: 1.35
`delivery`	Truck, District 1, 7–19h: 1.80

6.2.3 Tier 3a — Node-Dependent (Per Frontier Node)

$$\mathcal{N} = \{\texttt{intersection_density}, \texttt{road_works}, \texttt{markets}, \texttt{heuriger}\}$$

Intersection Density Let $\deg(n) = |\mathrm{adj}(n)|$, $s(n) = 1$ if traffic signal:

$$m_{\text{inter}}(n) = \begin{cases} 1 & \deg(n) \le 2 \\ \big[1 + 0.05(\deg(n) - 2) + 0.10\, s(n)\big] \cdot Z(D_n) & \deg(n) > 2 \end{cases}$$

Zone penalty: $Z(D = 1) = 1.30$, $Z(D \in \{7, 8\}) = 1.20$, $Z(D = 6) = 1.15$, else 1.0.

Markets / Heuriger Geospatial proximity check. If a Naschmarkt or Grinzing wine tavern is within 200–400 m on the active day/hour, apply ×1.45 or ×1.35.

6.2.4 Tier 3b — Edge-Dependent (Cheapest Outgoing Edge Proxy)

$\mathcal{D} = \{$`surface`, `elevation`, `safety`, `emission_zones`, `tram_tracks`, `snow_priority`, `commuter_bridges`, `lane_`

> **Why the cheapest outgoing edge proxy?.** $h(n)$ must *not* overestimate real cost. We don't know which edge A^* will take next. Taking the minimum across all outgoing edges gives a **lower bound** on edge quality, preserving admissibility. An average or maximum would overestimate.

```python
adj = graph["adjacency"].get(node_id, [])
best_edge_mult = float("inf")
for nb in adj:
    edge = graph["edges"][nb["edge_idx"]]
    e_mult = 1.0
    if _enabled(enabled, "surface"):
        e_mult *= get_surface_multiplier(edge, vehicle)
    if _enabled(enabled, "elevation"):
        e_mult *= get_elevation_multiplier(edge, vehicle)
    # ... (10 more edge heuristics)
    if e_mult < best_edge_mult:
        best_edge_mult = e_mult
if best_edge_mult != float("inf"):
    mult *= best_edge_mult
```

Listing 4: combined.py — edge-dependent tier

7 All 30 Heuristic Functions — Reference

7.1 Environmental (5)

- **weather**: Lookup table — clear 1.0, rain 1.50, black_ice 2.00. Black ice inferred: temp<2°C and humidity>80% upgrades clear/fog → black_ice automatically.
- **visibility**: Fog/mist penalty — <300 m visibility: ×1.60.
- **humidity**: Exposed vehicles (walking/bicycle/escooter) on high humidity (>85%): ×1.10.
- **wind**: Headwind component for bicycle/walking/escooter: $m = 1 + 0.03H_w$ (bicycle).
- **season**: Summer tourists (Jun–Aug, D1/6/7): ×1.15; Advent (Nov–Dec): ×1.25.

7.2 Temporal (5)

- **time_of_day**: Rush hours — AM rush (7–9): ×1.60; PM rush (17–19): ×1.70.
- **holiday_historical**: 13 Austrian holidays in 2026; D1 on holiday: ×1.20; outer: ×0.80.
- **events**: Scheduled events with zones/times — Marathon: ×2.50, Opernball: ×1.60.
- **headway**: Transit service frequency — metro off-peak 7 min wait: ×1.14.
- **nightnetwork**: Bus/metro at 0–5 AM: ×1.30 (reduced service).

7.3 Vehicle-Type (3)

- **vehicle_type**: Speed baseline — motorcycle: 0.85, walking: 1.80, metro: 0.40.
- **vehicle_type_routing**: Hard passability filter (removes edges entirely).
- **delivery**: Truck, District 1, daytime: $\times 1.80$ (delivery restriction zones).

7.4 Road Geometry & Condition (6)

- **surface**: Cobblestone bicycle: $\times 1.60$; gravel bicycle: $\times 1.80$; escooter on gravel: $\times 2.00$.
- **elevation**: Uphill slope (%) for walking/bicycle/bus: $m = 1 + 0.05 \cdot \text{slope}$.
- **lane_capacity**: ≥ 4 lanes for bulk vehicles: $\times 0.90$ (incentive); 1 lane: $\times 1.10$.
- **tram_tracks**: Bicycle on tram track + wet: $\times 1.70$; dry: $\times 1.25$.
- **snow_priority**: Non-priority road + snow: $\times 1.60$; priority road: $\times 1.05$.
- **road_quality**: Composite of road type $\times$ surface $\times$ speed limit — steps: $\times 2.50$.

7.5 Safety & Regulation (6)

- **safety**: Pedestrian, night (21h–5h), unlit road: $\times 1.50$; lit primary: $\times 0.90$.
- **school_zones**: School edge, 7:30–8:15 or 13:00–14:00 weekdays: $\times 1.35$.
- **emission_zones**: Motor vehicle on restricted road (`motor_vehicle=no`): $\times 8.0$ (soft ban).
- **commuter_bridges**: Praterbrücke, rush hour weekday: $\times 2.00$; Reichsbrücke: $\times 1.80$.
- **road_works**: Within 150 m of active construction: $\times 1.50$.
- **manual_adjustment**: User slider $I \in [0, 100]$: $m = 0.5 + 0.025I$, range $[0.5, 3.0]$.

7.6 Urban Features & Attractions (5)

- **intersection_density**: High degree + signal + District 1: up to $\sim \times 1.9$.
- **pedestrian_density**: District 1, daytime: $\times 1.35$; on holiday $\times 1.35 \cdot 1.15$.
- **markets**: Within 200 m of Naschmarkt or farmers' market on operating day: $\times 1.45$.
- **heuriger**: Grinzing wine taverns, Sep–Oct Fri–Sat 16–23h: $\times 1.35$.
- **fiaker**: Horse-carriage zone (D1, 10–18h), car/bicycle: $\times 1.25$.

7.7 Destination-Specific (2)

- **parking**: Car to District 1: $\times 1.60$; District 2: $\times 1.35$; outer: $\times 1.05$.
- **scenic**: `route_profile=greenest`, near green space: $\times 0.85$ (incentive); not green: $\times 1.05$.

Maximum multipliers ranked. `emission_zones`: 8.0 (soft-ban) → `weather+walking (thunderstorm)`: 4.60 → `manual`: 3.0 → `events (Marathon)`: 2.50 → `commuter_bridges`: 2.00 → `black_ice`: 2.00

8 Admissibility, Capping, and ε-Admissibility

8.1 What is Admissibility?

A heuristic is **admissible** when it *never overestimates* true cost:

$$h(n) \leq h^*(n) \quad \forall n$$

Pure Haversine distance is admissible (straight-line $\leq$ any road path). But multiplying it by factors > 1 (rain, rush hour) can make it inadmissible.

8.2 The MAX_HEURISTIC_CAP

$$h_{\text{final}}(n) = \min\big(h_{\text{raw}}(n),\ d_0(n, g) \cdot C\big), \quad C = 50$$

Why 50?. Without a cap, stacked multipliers can reach 10^3+ (thunderstorm × tram tracks × cobblestone × rush hour × ...). At those values, $h(n)$ completely dominates $g(n)$, and A* degenerates to a greedy search — losing its optimality guarantee. Cap = 50 means the heuristic claims at most a 50× cost blowup, keeping weighted A* ε-admissible.

8.3 ε-Admissibility (Weighted A*)

Weighted A* uses $f = g + w \cdot h$. Even with $h > h^*$, if $h \leq w \cdot h^*$ the solution is bounded:

$$\text{cost}(\pi) \ \leq \ w \ \cdot \ \text{cost}(\pi^*)$$

Default weight: $w = 1.5$, cap $C = 50$, so paths are at most 1.5× optimal.

9 The 8 Search Algorithms

9.1 How All Algorithms Share the Same Interface

```
def find_path(graph, start_id, goal_id, heuristic_fn, params) -> PathResult
```

Listing 5: Every algorithm has this signature

All algorithms:

1. Call `is_edge_blocked()` — skip intensity≥95 edges
2. Call `is_edge_passable()` — skip vehicle-incompatible edges
3. Call `get_effective_edge_cost()` — apply manual overrides + road quality to g
4. Return a `PathResult` with `path_coords`, `distance_m`, `nodes_expanded`, `compute_time_ms`

9.2 Algorithm 1: BFS (Breadth-First Search)

Data structure: `deque` (FIFO queue). Explores all distance-1 neighbours, then distance-2, etc.

Optimality: Finds *minimum-hop* path, **not** minimum-distance. On Vienna's graph (edges 30–300 m), hop-count ≠ distance.

Heuristic used: None — ignores $h(n)$.

Typical: 800–1 000 nodes expanded, 50–100 ms, path ≈50% longer than optimal.

9.3 Algorithm 2: DFS (Depth-First Search)

Data structure: Python list as stack (LIFO). Explores deep first, backtracks.

Optimality: Not optimal. Path depends on neighbour iteration order; can find near-worst route.

Heuristic used: None.

Typical: 600–1 000 nodes expanded, highly variable path length.

9.4 Algorithm 3: UCS (Uniform-Cost Search)

Data structure: `heapq` priority queue keyed on $g(n)$. Allows re-expansions (stale entry check).

Optimality: **Optimal** for non-negative edge weights. Used as **ground truth** for optimality gap calculations.

Difference from Dijkstra: No visited set. Stale entry detection at line 35–36: `if g > dist[node]: continue`.

Heuristic used: None (only $g(n)$).

9.5 Algorithm 4: Dijkstra

Data structure: `heapq` + explicit `visited` closed set. Each node expanded **exactly once**.

Optimality: **Optimal**. Slightly more efficient than UCS (no re-expansions).

Role in project: **Definitive ground truth** — runner computes `optimality_gap_pct = (other - dijkstra) / dijkstra × 100` for all 8 results.

Heuristic used: None.

Typical: 300–500 nodes expanded, 15–30 ms.

9.6 Algorithm 5: Greedy Best-First Search

Data structure: `heapq` keyed on $h(n)$ only (ignores g). Always expands node closest to goal by heuristic estimate.

Optimality: **Not optimal**. Greedy choice can lead to dead-ends or suboptimal routes.

Heuristic: `make_heuristic(params)` closure — all 3 tiers.

How heuristic affects it: The heuristic is the *only* decision factor. Strong heuristics give fast, decent routes; poor heuristics give terrible routes. No g-cost feedback.

Typical: 50–150 nodes expanded, 5–10 ms, path 10–30% longer than optimal.

9.7 Algorithm 6: A*

Data structure: `heapq` keyed on $f(n) = g(n) + h(n)$. Closed set prevents re-expansion.

Optimality: **Optimal** when h is admissible.

How heuristic affects it: $h(n)$ guides the search toward the goal while $g(n)$ keeps it honest about actual cost. Bad heuristic → explores many nodes (degrades to Dijkstra). Good heuristic → explores far fewer.

```
f = tentative_g + heuristic_fn(neighbor, goal_id, graph, params)
heapq.heappush(open_set, (f, tiebreak, neighbor))
```

Listing 6: astar.py — core loop

Typical: 200–400 nodes expanded, 10–20 ms, **optimal path**.

9.8 Algorithm 7: Weighted A*

Data structure: Same as A* but $f(n) = g(n) + w \cdot h(n)$ where $w = 1.5$ (default).

Optimality: **Bounded suboptimal**: cost $\leq w\times$ optimal. Cap ensures this holds.

Weight tuning:

- $w = 1.0 \rightarrow$ A* (optimal, slower)
- $w = 1.5 \rightarrow$ default, good speed/quality trade-off
- $w = 2.0+ \rightarrow$ very fast, path quality degrades
- $w \rightarrow \infty \rightarrow$ approaches greedy best-first

How heuristic affects it: Amplified by w — a $1.5\times$ weather multiplier in h now reads as $1.5 \times w = 2.25\times$. Heuristic dominance increases with w.

Typical: 50–100 nodes expanded, 5–10 ms.

9.9 Algorithm 8: Bidirectional A*

Idea: Run A* forward (start→goal) *and* backward (goal→start) simultaneously. Expand whichever side has smaller top-of-heap f.

Meeting condition:

$$\text{best_sum} = \min_{n:\, n \in \text{fwd} \cap \text{bwd}} \big(g_f(n) + g_b(n)\big)$$

Early termination: When top_fwd + top_bwd $\geq$ best_sum, the best meeting is provably optimal.

Reverse adjacency: `_reverse_adjacency(node, graph)` iterates over node's neighbours' neighbours to find predecessors. Uses `edge_idx` of the forward edge for cost/block checks.

```python
top_f = open_f[0][0] if open_f else float("inf")
top_b = open_b[0][0] if open_b else float("inf")
if top_f + top_b >= best_sum:
    break  # provably optimal
if top_f <= top_b:
    # expand forward
else:
    # expand backward
```

Listing 7: bidirectional_astar.py — expansion decision

Optimality: **Optimal** if h admissible in both directions.
Typical: 50–100 nodes total (fwd + bwd combined), 2–5× faster than A*.

10 How Heuristics Affect Each Algorithm

Algorithm	Uses $h(n)$?	Uses $g(n)$?	Effect of heuristic
BFS	No	No	Heuristic irrelevant; hop-count search
DFS	No	No	Heuristic irrelevant; stack order determines path
UCS	No	Yes	Pure cost-driven; heuristic has no effect
Dijkstra	No	Yes	Pure cost-driven; ground truth baseline
Greedy	h only	No	Entire route determined by heuristic quality
A*	$f = g + h$	Yes	Heuristic guides and speeds up optimal search
Weighted A*	$f = g + wh$	Yes	Amplified heuristic; faster but bounded-suboptimal
Bidirectional A*	$f = g + h$ (both dirs)	Yes	Heuristic guides meeting point; bidirectional pruning

Practical implication. Toggle weather → *rain* and watch Greedy change routes dramatically (heuristic dominates), while UCS/Dijkstra routes barely change (only edge costs shift slightly via road quality). This demonstrates the direct influence of heuristic weighting.

11 Parallel Runner and PathResult

11.1 run_all()

```python
def run_all(start_lat, start_lon, goal_lat, goal_lon, params, heuristic_fn):
    graph = load_graph()
    start_id = nearest_node(graph, start_lat, start_lon)
    goal_id  = nearest_node(graph, goal_lat, goal_lon)

    with ThreadPoolExecutor(max_workers=8) as ex:
        futures = {
            "bfs":                ex.submit(bfs.find_path, ...),
            "dijkstra":           ex.submit(dijkstra.find_path, ...),
            # ... all 8
        }
    results = {name: f.result() for name, f in futures.items()}

    # Optimality gap vs. Dijkstra
    dijkstra_dist = results["dijkstra"]["distance_m"]
    for name, r in results.items():
        r["optimality_gap_pct"] = (r["distance_m"] - dijkstra_dist) / dijkstra_dist
     * 100
    return results
```

Listing 8: src/algorithms/runner.py — parallel dispatch

11.2 PathResult Fields

Field	Meaning
`path_node_ids`	List of node ID strings start→goal
`path_coords`	List of (lat, lon) tuples for Leaflet
`distance_m`	Sum of raw edge `distance_m` values
`estimated_time_s`	`distance_m` / `VEHICLE_SPEEDS[vehicle]`
`nodes_expanded`	Count of nodes popped from queue/heap
`compute_time_ms`	Wall-clock time (`perf_counter`)
`is_optimal`	True if matches Dijkstra distance
`optimality_gap_pct`	% above Dijkstra optimal
`error`	None if found; error string if not

Default speed: `car` = 8.33 m/s (≈30 km/h urban). Distance/speed = estimated travel time in seconds.

12 Closure Factory Pattern — Performance

12.1 Why a Closure?

A* calls `h(node, goal, graph, params)` thousands of times per route. 8 algorithms run in parallel. With ≈300 nodes × 8 algorithms = 2 400 heuristic calls per request, every unnecessary multiplication counts.

```python
def bad_heuristic(node, goal, graph, params):
    mult  = get_weather_multiplier(params["weather"], params["vehicle"])  # every call!
    mult *= get_time_multiplier(params["hour"], params["day_of_week"])     # every call!
    # ... 6 more static calls per invocation
```

Listing 9: Without closure — SLOW (recomputes static tier every call)

```python
def make_heuristic(params):
    static_mult = 1.0
    static_mult *= get_weather_multiplier(weather, vehicle)   # once
    static_mult *= get_time_multiplier(hour, day_of_week)     # once
    # ...
    def heuristic(node_id, goal_id, graph, _params=None):
        mult = static_mult                                     # free read
        # dynamic tier here
        h = h_base * mult
        return min(h, h_base * MAX_HEURISTIC_CAP)
    return heuristic
```

Listing 10: With closure — FAST (static tier baked in)

Static tier = 8 multiplications done *once*. Saves ≈8 × 2 400 = 19 200 extra multiplications per request.

12.2 enable/disable via UI

```python
def _enabled(enabled: set[str], key: str) -> bool:
    # Empty set = all active (default: all 30 on)
    # Non-empty = strict whitelist
    return not enabled or key in enabled
```

Listing 11: combined.py — $_{e}nabled()helper$

13 Fully Worked Heuristic Calculation Example

Scenario: Bicycle commute, Tuesday 17:00, rain ($T = 10C$, $H = 80\%$, $V = 2000$ m, wind 5 m/s headwind), destination District 1, `route_profile=fastest`, all heuristics on. Frontier node n has degree 4, no signal, on cobblestone residential edge with tram track, slope +3%, District 1.

Tier $\mathcal{S}$ (computed once):

$$\begin{aligned}
m_{\text{weather}} &= 1.50 \cdot 1.60 = 2.40 \quad \text{(rain base} \times \text{bicycle rain factor)} \\
m_{\text{time}} &= 1.70 \quad \text{(weekday PM rush 17–19)} \\
m_{\text{vehicle}} &= 1.20 \quad \text{(bicycle)} \\
m_{\text{vis}} &= 1.15 \quad (1000 \leq 2000 < 3000) \\
m_{\text{wind}} &= 1 + 0.03 \cdot 5 = 1.15 \\
M_{\mathcal{S}} &= 2.40 \times 1.70 \times 1.20 \times 1.15 \times 1.15 \approx 6.47
\end{aligned}$$

Tier $\mathcal{G}$:

$$\begin{aligned}
m_{\text{fiaker}} &= 1.25 \quad (D1, \text{bicycle}, h \in [10, 18)) \\
m_{\text{ped}} &= 1.35 \quad (D1, \text{bicycle}, h \in [11, 20)) \\
M_{\mathcal{G}} &= 1.25 \times 1.35 = 1.6875
\end{aligned}$$

Tier $\mathcal{N}$:

$$\begin{aligned}
m_{\text{inter}} &= [1 + 0.05 \times (4 - 2)] \times 1.30 = 1.10 \times 1.30 = 1.43 \\
M_{\mathcal{N}} &= 1.43
\end{aligned}$$

Tier $\mathcal{D}$ (edge proxy):

$$\begin{aligned}
m_{\text{surf}} &= 1.60 \quad \text{(cobblestone, bicycle)} \\
m_{\text{elev}} &= 1 + 0.05 \times 3 = 1.15 \\
m_{\text{tram}} &= 1.70 \quad \text{(bicycle + wet)} \\
m_{\text{rq}} &= \text{clamp}[0.4, 2.5,\ 1.00 \times 1.30 \times 1.00 \times 1.25] = 1.625 \\
M_{\mathcal{D}} &= 1.60 \times 1.15 \times 1.70 \times 1.625 \approx 5.08
\end{aligned}$$

Combined:

$$M = 6.47 \times 1.6875 \times 1.43 \times 5.08 \approx 79.4$$

Cap kicks in: $\min(79.4,\ 50) = 50$.

$$h(n, g) = 50 \cdot d_0(n, g)$$

If $d_0 = 1200$ m, then $h = 60\,000$ m — interpreted as equivalent to 60 km in routing cost, reflecting compounded rush-hour + rain + cobblestone + tram + dense intersection + pedestrian zone.

14 Algorithm Performance Summary

Table 1: Typical performance on Vienna graph (interdistrict route ≈2 km, 1 000 nodes)

Algorithm	Optimal?	Uses h?	Time (ms)	Nodes expanded	Path (m)	vs Optimal
BFS	No (hops)	No	50–100	800–1000	≈2500	+50%
DFS	No	No	30–80	600–1000	Variable	Variable
UCS	Yes	No	15–30	300–500	≈1800	Ground truth
Dijkstra	Yes	No	15–30	300–500	≈1800	Ground truth
Greedy	No	Yes	5–10	50–150	≈2000	+10%
A*	Yes	Yes	10–20	200–400	≈1800	Optimal
Weighted A*	Bounded	Yes	5–10	50–100	≈1900	+5%
Bidirectional A*	Yes	Yes	8–15	50–100	≈1800	Optimal

15 Quick Q&A: Likely Professor Questions

Q: Why is the heuristic a closure? Static multipliers (weather, time, vehicle) depend only on the request parameters, not on which node we're evaluating. Computing them once and capturing in a closure avoids ≈19 000 redundant multiplications per request.

Q: How do you ensure admissibility? Haversine is always $\leq$ true road distance (triangle inequality). The edge proxy picks the cheapest outgoing edge (lower bound on edge quality). The hard cap (50) prevents inadmissible blow-up. Together: $h(n) \leq h^*(n)$ for the static+edge tiers; dynamic tiers add multipliers ≥ 1 but the cap bounds the damage.

Q: Why does bidirectional A* expand fewer nodes? Forward search from A expands a sphere of radius r around A. Backward search from B expands a sphere of radius r around B. They meet when $r_A + r_B \approx d(A, B)$, so each sphere has radius $\approx d/2$ instead of d. Volume of two half-spheres $\approx$ half the volume of one full sphere.

Q: What is the optimality gap? `optimality_gap_pct = (algo_dist - dijkstra_dist) / dijkstra_dist × 100`. A gap of 0 means the algorithm found the same path as Dijkstra. A gap of 5% means the algorithm's route is 5% longer than optimal.

Q: Why does UCS differ from Dijkstra? UCS allows re-expansion (stale entry check). Dijkstra uses a visited closed-set — each node expanded exactly once, never revisited. For non-negative edge weights, both produce optimal paths. Dijkstra is slightly more cache-efficient; UCS is simpler to implement.

Q: What happens when intensity = 95 on an edge? `get_effective_edge_cost()` returns `float("inf")`. All 8 algorithms check `is_edge_blocked()` before considering any neighbour — the edge is simply never explored. Metro/train routes return empty paths for the same reason (no passable edges at all).

Q: Why multiply, not add? Multiplicative: weather 1.5 × cobblestone 1.6 = 2.4 (realistic: bad surface in bad weather is compounding). Additive: 0.5 + 0.6 = 1.1 (barely noticeable). Multiplication correctly models independent cost factors that each slow you proportionally.

Q: How does a heuristic become a scenario? Each scenario preset (e.g. "Viennese Marathon Day") sets specific param values: `date=2026-04-26`, `hour=9`, enabling the `events` heuristic. The marathon event record fires with ×2.50 for zones 1–4 and 9, routing algorithms around the closed streets.

16 Summary of All Heuristic Functions (Tier Classification)

Table 2: All 30 heuristics, tier, module, and max multiplier

ID	Tier	Module	Max mult.
weather	$\mathcal{S}$ static	weather.py	4.60 (thunder+walking)
time_of_day	$\mathcal{S}$ static	time_of_day.py	1.70 (PM rush weekday)
vehicle_type	$\mathcal{S}$ static	vehicle_type.py	1.80 (walking)
visibility	$\mathcal{S}$ static	visibility.py	1.60 (<300 m fog)
humidity	$\mathcal{S}$ static	humidity.py	1.10
nightnetwork	$\mathcal{S}$ static	nightnetwork.py	1.30
wind	$\mathcal{S}$ static	wind.py	∼1.15
headway	$\mathcal{S}$ static	headway.py	1.60 (train off-peak)
parking	$\mathcal{G}$ goal	parking.py	1.60 (D1, car)
holiday_historical	$\mathcal{G}$ goal	holiday_historical.py	1.20 / 0.80
events	$\mathcal{G}$ goal	events.py	2.50 (Marathon)
fiaker	$\mathcal{G}$ goal	fiaker.py	1.25
season	$\mathcal{G}$ goal	season.py	1.25 (Advent)
pedestrian_density	$\mathcal{G}$ goal	pedestrian_density.py	1.55 (D1 holiday)
delivery	$\mathcal{G}$ goal	delivery.py	1.80 (truck D1)
intersection_density	$\mathcal{N}$ node	intersection_density.py	∼1.9
road_works	$\mathcal{N}$ node	road_works.py	1.50
markets	$\mathcal{N}$ node	markets.py	1.45
heuriger	$\mathcal{N}$ node	heuriger.py	1.35
surface	$\mathcal{D}$ edge	surface.py	2.00 (escooter gravel)
elevation	$\mathcal{D}$ edge	elevation.py	varies (slope)
safety	$\mathcal{D}$ edge	safety.py	1.50 / 0.90
emission_zones	$\mathcal{D}$ edge	emission_zones.py	8.00 (soft ban)
tram_tracks	$\mathcal{D}$ edge	tram_tracks.py	1.70 (bike wet)
snow_priority	$\mathcal{D}$ edge	snow_priority.py	1.60
commuter_bridges	$\mathcal{D}$ edge	commuter_bridges.py	2.00 (Praterbrücke)
lane_capacity	$\mathcal{D}$ edge	lane_capacity.py	1.10 / 0.90
school_zones	$\mathcal{D}$ edge	school_zones.py	1.35
scenic	$\mathcal{D}$ edge	scenic.py	1.05 / 0.85
road_quality	$\mathcal{D}$ edge	road_quality.py	2.50 (capped)
manual_adjustment	$\mathcal{D}$ edge	manual_adjustment.py	3.00